

# zk-PoP: Zero-Knowledge Proof of Proof

Applied to the Lean 4 port of the Octra lite node

Version 1

## Abstract

A formal development is only as convincing as the artifact that backs it. The usual way to back it is to publish the proof and let anyone re-check it, but a proof is often the thing one does not wish to publish: it may encode a construction, a strategy, a private dataset, or simply an asset whose value lies in not being copied. This paper specifies *zk-PoP*, a protocol by which a prover convinces a verifier that it holds a checked proof of a published statement while revealing only a handful of the proof's steps, and specifies it precisely enough that the protocol's guarantees are theorems rather than claims. Completeness, soundness against a committed prover, the numerical soundness error, and the privacy of the transcript are all proved in Lean 4 in the accompanying development `RequestProject.ZkPoP`; the statements are reproduced here alongside the exact hypotheses they need, and the scheme is instantiated on the Lean 4 port of the Octra lite node's formal models. What is *not* proved is stated as plainly as what is: the two cryptographic idealisations, the committed-prover model of soundness, and the fact that the error bound proved here is the bound for spot-checking an unencoded proof.

## 1 Introduction

The Octra lite node ships four machine-checked models of the protocol's core invariants. Those models have been transcribed to Lean 4 and re-proved, and the port carries with it a much larger body of formalised material about the node, the network and the litepaper. The port's claim to be worth anything is the claim that a specific set of statements is proved, in a specific kernel, from a specific set of axioms.

There are two conventional ways to substantiate such a claim. The first is to publish everything and let a third party re-run the checker. This is the gold standard, and it is what the port does. The second is to publish nothing and ask to be believed. Between the two there is a gap that shows up as soon as a proof is treated as an asset: a prover wants to sell, license, stake or be paid for a proof, and must therefore demonstrate that the proof exists and checks *before* handing it over — or without handing it over at all.

zk-PoP fills that gap. It is a *proof of proof*: a protocol whose subject matter is not a mathematical statement directly but the existence of a checked derivation of one. The design is deliberately unexotic — commit to the derivation, let the verifier spot-check it, keep the unopened part hidden — because the point of this document is not novelty but exactness. Every claim made about the protocol below corresponds to a theorem in the Lean development `RequestProject.ZkPoP`, and the correspondence is tabulated in Appendix A and in `ZKPOP.md`.

**Scope.** This paper specifies the interactive, honest-verifier protocol and its properties. Section 9 lists what the formalisation assumes and what it leaves open, including the Fiat–Shamir compilation to a non-interactive proof and the proximity amplification that a deployment needs in order to make the query count practical.

## 2 Setting and threat model

**Parties.** A prover  $P$  holds a derivation  $\pi$  of a judgement  $j$ . A verifier  $V$  holds  $j$  and the public parameters.  $V$  interacts with  $P$  and outputs accept or reject.

**What the verifier should learn.** On acceptance,  $V$  should be convinced that a derivation of  $j$  exists under the published rules. It should learn nothing else about  $\pi$  beyond the steps it explicitly opens.

**Adversary.** A dishonest prover is an arbitrary function from challenges to answers. It is *committed*: it publishes its commitment before seeing the challenge, and this is the only restriction placed on it. A dishonest prover that exhibits a collision in the hash function or in the leaf commitment is outside the model — soundness is stated conditionally on no such collision, which is the usual reduction. A dishonest verifier is not modelled here: the privacy statement of Section 7 is for the honest-verifier setting, which suffices for the Fiat–Shamir compilation.

**Notation.**  $\mathcal{J}$  is a type of judgements,  $R$  a published rule table,  $H$  a two-to-one compression function on digests,  $n$  the number of steps in a derivation,  $k$  the number of positions the verifier queries, and  $d$  the depth of the Merkle tree, with  $n \leq 2^d$ .

## 3 Proofs as locally checkable traces

The verifier will see a few steps of a derivation, never the whole of it. For that to carry information, “being a proof” must be a conjunction of local conditions.

**Definition 3.1** (Rule table). A rule table over  $\mathcal{J}$  is a decidable predicate  $R.\text{ok}(ps, c)$  on a finite premise list  $ps$  and a conclusion  $c$ : it holds exactly when  $ps \vdash c$  is an instance of a published inference rule. (Rules)

**Definition 3.2** (Trace). A step is a pair (premise list, conclusion) in which each premise is tagged with the position of the earlier step that derives it. A trace is a finite list of steps. (Step, Trace)

**Definition 3.3** (Local check). Position  $i$  of a trace  $t$  passes the local check when the step  $s$  stored there exists,  $R.\text{ok}$  holds of its premises and conclusion, and every premise of  $s$  is tagged with a position  $r < i$  at which a step with exactly that conclusion is stored. A trace is valid when every position passes. (Trace.StepOk, Trace.Valid)

The local check reads the step at  $i$  and the steps it references, and nothing else. That is what makes it a legal question for the verifier to ask, and the next theorem is what makes the answer worth having.

**Theorem 3.4** (Local checks imply derivability). If every position of  $t$  passes the local check, then every conclusion appearing in  $t$  is derivable from  $R$  alone. In particular, if the last step of a valid trace concludes  $j$ , then  $j$  is derivable. (Trace.derivable\_of\_valid, Trace.derivable\_of\_proves)

Derivability here is the inductive predicate generated by the rule table, with no reference to traces, positions or commitments; the theorem is proved by strong induction on the position, using that premises are discharged by strictly earlier steps.

The contrapositive is the form the protocol uses.

**Theorem 3.5** (What spot-checking hunts for). If  $j$  is not derivable, then every trace whose last step concludes  $j$  fails the local check at some position. (Trace.exists\_badIndex)

Finally, the local check is a program, not merely a predicate: the development gives a Bool-valued checker and proves it equivalent to the specification (`Trace.step0kB_iff`, `Trace.validB_iff`). The instantiation of Section 8 runs it inside the kernel.

## 4 Commitments

### 4.1 Merkle trees

The prover publishes a single digest for its whole trace and later opens positions of it. An *authentication path* is a list of (side, sibling) pairs, from the leaf upwards; recomputing the root from a leaf and a path is written  $\text{pathRoot}_H$ . An opening of position  $\text{addr}$  to leaf  $x$  with path  $p$  is *accepted* against root  $rt$  when the sides recorded in  $p$  are the address bits of  $\text{addr}$  and  $\text{pathRoot}_H(p, x) = rt$  (`Opens`).

**Assumption 4.1** (Collision freedom).  *$H$  has no collisions:  $H(a, b) = H(c, d)$  implies  $(a, b) = (c, d)$ . Equivalently  $H$  is pairwise injective (`hashCollisionFree_iff`).*

**Theorem 4.2** (Position binding). *Under Assumption 4.1, two accepted openings of the same address against the same root expose the same leaf. (`open_unique`)*

**Theorem 4.3** (Opening completeness). *In the perfect tree of depth  $d$  over leaves  $\ell_0, \dots, \ell_{2^d-1}$ , every position  $i < 2^d$  has an accepted opening exposing  $\ell_i$ . (`Tree.exists_opens_perfect`, `exists_acceptsAt`)*

Theorem 4.3 is proved from an explicit construction: the tree, the bit address of an index, the authentication path for an address, and the lemma that the honest path recomputes the root (`Tree.pathRoot_proof`).

### 4.2 Leaf commitments

A leaf digest commits to one step under a nonce,  $\text{leafOf}(s, \nu)$ . Two idealisations are available for it, and they are used in different places.

**Assumption 4.4** (Leaf binding). *No two different steps share a leaf digest (`LeafScheme.Collision` is false).*

**Assumption 4.5** (Perfect hiding). *Every leaf digest is compatible with every step: for all  $s, s', \nu$  there is  $\nu'$  with  $\text{leafOf}(s, \nu) = \text{leafOf}(s', \nu')$  (`LeafScheme.PerfectlyHiding`).*

### 4.3 Why the two are never assumed together

**Theorem 4.6.** *A leaf scheme satisfying Assumptions 4.4 and 4.5 simultaneously can commit to only one step: all steps are equal. (`LeafScheme.subsingleton_of_hiding_of_no_collision`)*

This is the familiar fact that a commitment cannot be perfectly hiding and perfectly binding at once, and it is stated here as a theorem for a practical reason: it forces the development to keep two theorem families apart. Soundness (Sections 5–6) is proved under Assumptions 4.1 and 4.4; privacy (Section 7) is proved under Assumption 4.5. No theorem in the development assumes both, so no theorem in the development is vacuous for that reason. A deployment picks one idealisation to be perfect and the other to be computational, exactly as it must for any commitment scheme.

## 5 The protocol

**Public parameters** (`Params`): the rule table  $R$ , the compression function  $H$ , the leaf scheme  $L$ , the tree depth  $d$ , and a padding step for the unused leaves.

**Commit.** The prover holding a trace  $t$  of length  $n \leq 2^d$  and nonces  $\nu$  publishes

$$\text{root} = \text{root}_H(i \mapsto \text{leafOf}(t_i, \nu_i)),$$

the Merkle root of the per-step commitments (`Params.commitTrace`).

**Challenge.** The verifier sends a list of positions  $q_1, \dots, q_k < n$ .

**Response.** The prover opens each queried position *and* each position referenced by a queried step, sending for every opened position the step, its nonce, and its authentication path (`Params.openSet`, `Params.respond`).

**Verify** (`Params.Accepts`): every opening is accepted against the published root, and every queried position passes the local rule check on the opened material — the step is a rule instance, and each of its premises is the conclusion of an opened, strictly earlier step.

## 5.1 Completeness

**Theorem 5.1** (Completeness). *If  $t$  is valid,  $n \leq 2^d$ , and every queried position is below  $n$ , then the honest transcript is accepted, for every challenge and every choice of nonces. (`Params.accepts_transcript`)*

## 5.2 Soundness

Soundness is stated against a prover that has committed to a trace: the root it published is the root of some list  $t$  of steps under some nonces. Nothing else about the prover is assumed — in particular its answers may be arbitrary functions of the challenge.

**Theorem 5.2** (The opened step is the committed step). *Under Assumptions 4.1 and 4.4, an accepted opening of position  $i < n$  against root exposes exactly the step  $t_i$ . (`Params.opened_step_eq`)*

**Theorem 5.3** (Soundness at a queried position). *Under the same assumptions, if the verifier accepts and position  $i$  was queried and lies inside the committed trace, then the committed trace really passes its local check at  $i$ . (`Params.stepOk_of_accepts`)*

Combining Theorems 3.5 and 5.3:

**Theorem 5.4** (A liar must be lucky). *If the judgement claimed by the committed trace has no derivation, then the trace has a position  $i_0$  failing its local check, and on every challenge the verifier accepts,  $i_0$  was not queried. (`Params.exists_unqueried_bad_index`)*

## 6 The soundness error

Let the challenge be a  $k$ -tuple of positions drawn from the  $n$  positions of the committed trace.

**Lemma 6.1.** *The  $k$ -tuples avoiding one forbidden position number exactly  $(n-1)^k$ . (`card_filter_avoiding`)*

**Theorem 6.2** (Soundness error). *Under Assumptions 4.1 and 4.4, let a committed prover claim a judgement with no derivation. Then, whatever it answers, the set of challenges on which the verifier accepts has at most  $(n-1)^k$  elements out of  $n^k$ ; as a probability over a uniform challenge,*

$$\Pr[\text{accept}] \leq \left(1 - \frac{1}{n}\right)^k.$$

(`cheating_prover_card_le`, `cheating_prover_prob_le`)

The prover is quantified as an arbitrary function from challenges to transcripts, so the bound is not a statement about a particular cheating strategy.

| $n$    | $k$            | $(1 - 1/n)^k$        | error in bits |
|--------|----------------|----------------------|---------------|
| $10^3$ | $10^3$         | $3.68 \cdot 10^{-1}$ | 1.4           |
| $10^3$ | $10^4$         | $4.52 \cdot 10^{-5}$ | 14.4          |
| $10^3$ | $2 \cdot 10^4$ | $2.04 \cdot 10^{-9}$ | 28.9          |
| $10^4$ | $10^4$         | $3.68 \cdot 10^{-1}$ | 1.4           |
| $10^4$ | $2 \cdot 10^4$ | $1.35 \cdot 10^{-1}$ | 2.9           |

Table 1: The bound of Theorem 6.2, evaluated numerically. These are arithmetic illustrations of the proved inequality, not additional results.

**Reading the bound.**  $(1 - 1/n)^k \approx e^{-k/n}$ : reaching a soundness error of  $2^{-\lambda}$  requires  $k \approx \lambda \ln 2 \cdot n$  queries, i.e. several tens of openings per step of the derivation. That is the honest price of spot-checking a proof that has *not* been encoded: a derivation can be wrong in exactly one place, and one place out of  $n$  is what a uniform query has to find. Deployments therefore commit not to the raw derivation but to a distance-amplifying encoding of it, so that an invalid object disagrees with every valid one in a constant fraction of positions and a constant number of queries suffices. That encoding step is standard and is *not* part of the present formalisation; Theorem 6.2 is the exact bound for the protocol as specified here, and it is stated in that form rather than in an optimistic one.

## 7 What the verifier learns

**Theorem 7.1** (Simulation). *Under Assumption 4.5, let  $t$  and  $t'$  be traces agreeing at every opened position. Then there are nonces  $\nu'$  for which the honest transcript of  $t$  under  $\nu$  and the honest transcript of  $t'$  under  $\nu'$  are equal as objects.* (*Params.transcript\_eq\_of\_agree*)

The statement is stronger than indistinguishability: the two transcripts are the same transcript. A simulator handed only the opened steps may complete them to any trace it likes and produce a transcript the verifier cannot tell from the real one, because it is the real one for some choice of nonces. Two consequences:

- *Witness indistinguishability.* Two different proofs of the same judgement that the queries cannot separate produce identical transcripts (*Params.witness\_indistinguishable*).
- *Bounded leakage.* The prover never reveals more steps than there are opened positions — the queried ones and the positions they reference (*Params.card\_openings\_le*).

The mechanism is nonce re-randomisation (*Params.rerandomize*): on the opened positions the two traces agree, so the nonces are kept; elsewhere perfect hiding supplies a nonce explaining the same leaf digest with the other trace’s step. All published values — root, paths, openings — are functions of the leaf digests, and those are unchanged (*Params.leafFn\_rerandomize*).

**Remark 7.2.** *Theorem 7.1 hides everything except the opened steps; the opened steps themselves are shown to the verifier in the clear. Hiding those too is what the MPC-in-the-head or PCP-with-zero-knowledge compilations achieve, at the cost of committing to a randomised encoding rather than to the derivation itself. This paper specifies and proves the former; the latter is out of scope, and is not claimed.*

## 8 The scheme applied to the Octra lite node port

The layers above are generic in the judgement type and the rule table. Instantiating them on the port itself gives a concrete zk-PoP instance.

## 8.1 Declarations as judgements

Take judgements to be the fully qualified names of the port’s declarations, and take as rule table the port’s published dependency listing: the pair  $(c, ps)$  authorises concluding declaration  $c$  from exactly the declarations  $ps$ , and  $(c, [])$  records  $c$  as a base result of the ambient library (`depRules`). A trace over this table is a topologically ordered listing of declarations, each accompanied by the positions of the declarations it uses; a locally checked trace is a proof that the target declaration is grounded in the published listing.

## 8.2 A checked instance

`octraDepTable` is a listing taken from the port’s own dependency data for the self-boot kernel, and `selfBootTrace` is a ten-step trace over it ending in the kernel’s convergence theorem. The executable checker of Section 3 is run on it inside the Lean kernel:

```
Trace.validB (depRules octraDepTable) selfBootTrace = true    by decide,
```

and Theorem 3.4 converts that into derivability of the target (`selfBootTrace_validB`, `selfBootTrace_valid`, `kernelBootstrap_derivable`). Specialised to this instance, the protocol theorems read:

- *Completeness* — an honest prover of the port is accepted on every challenge inside the trace (`octra_honest_prover_accepted`);
- *Soundness* — a prover committing to a trace whose target is not grounded in the published listing has a locally bad position that every accepted challenge misses (`octra_bad_index_unqueried`).

## 8.3 The claim bundle

A claim is a module together with a declaration in it (`Claim`); a bundle is a list of claims, and its aggregate digest is the right fold

$$\text{chainDigest}(c_1, \dots, c_m) = H(\text{enc}(c_1), H(\text{enc}(c_2), \dots H(\text{enc}(c_m), \text{seed}) \dots)).$$

**Theorem 8.1** (The digest pins the bundle). *If  $H$  has no collisions and the claim encoding is injective, then two bundles of the same length with the same aggregate digest are the same bundle.* (*chainDigest\_injective*)

So a verifier holding the digest knows exactly which claims were made, and a prover cannot later reinterpret its bundle. `octraLiteNodeBundle` is the bundle for the four upstream models the port transcribes — the emission schedule, AML value safety, the consensus event space, and resource attestations.

## 9 What is assumed, and what is not proved

1. *Cryptographic idealisations.* Soundness assumes no collision in  $H$  and none in the leaf scheme; privacy assumes perfect hiding. By Theorem 4.6 these cannot all be perfect at once, and the development never assumes them at once. Reductions to concrete hardness assumptions are not formalised.
2. *Committed prover.* Soundness is proved for a prover whose published root is the root of an actual list of steps. Extraction of that list from an arbitrary adversarial digest is the standard extractability assumption for Merkle commitments and is not formalised.
3. *Error bound.* Theorem 6.2 is the bound for spot-checking the unencoded derivation; the proximity amplification that makes  $k$  small is not part of this development (Section 6).

4. *Honest verifier.* Privacy is stated for the transcript of an honest challenge. Malicious-verifier zero knowledge and the Fiat–Shamir compilation to a non-interactive proof are not formalised.
5. *The rule table is a parameter.* Theorem 3.4 derives derivability *from the published rules*. Whether those rules faithfully describe a particular kernel, or a particular dependency listing faithfully describes a particular repository, is an auditing question that the protocol does not answer and does not pretend to.

## A Index of formal statements

Every entry is a theorem of the Lean 4 development in `RequestProject/ZkPoP/`, proved with no sorry and with no axioms beyond `propext`, `Classical.choice` and `Quot.sound`.

| Paper    | Lean name                                                      | Module                             |
|----------|----------------------------------------------------------------|------------------------------------|
| Thm. 3.4 | <code>Trace.derivable_of_valid</code>                          | <code>ZkPoP/Trace.lean</code>      |
| Thm. 3.5 | <code>Trace.exists_badIndex</code>                             | <code>ZkPoP/Trace.lean</code>      |
| §3       | <code>Trace.validB_iff</code>                                  | <code>ZkPoP/Trace.lean</code>      |
| Thm. 4.2 | <code>open_unique</code>                                       | <code>ZkPoP/Merkle.lean</code>     |
| Thm. 4.3 | <code>Tree.exists_opens_perfect</code>                         | <code>ZkPoP/Merkle.lean</code>     |
| Ass. 4.1 | <code>hashCollisionFree_iff</code>                             | <code>ZkPoP/Commitment.lean</code> |
| Thm. 4.6 | <code>LeafScheme.subsingleton_of_hiding_of_no_collision</code> | <code>ZkPoP/Commitment.lean</code> |
| Thm. 5.1 | <code>Params.accepts_transcript</code>                         | <code>ZkPoP/Protocol.lean</code>   |
| Thm. 5.2 | <code>Params.opened_step_eq</code>                             | <code>ZkPoP/Protocol.lean</code>   |
| Thm. 5.3 | <code>Params.stepOk_of_accepts</code>                          | <code>ZkPoP/Protocol.lean</code>   |
| Thm. 5.4 | <code>Params.exists_unqueried_bad_index</code>                 | <code>ZkPoP/Protocol.lean</code>   |
| Lem. 6.1 | <code>card_filter_avoiding</code>                              | <code>ZkPoP/Soundness.lean</code>  |
| Thm. 6.2 | <code>cheating_prover_prob_le</code>                           | <code>ZkPoP/Soundness.lean</code>  |
| Thm. 7.1 | <code>Params.transcript_eq_of_agree</code>                     | <code>ZkPoP/Privacy.lean</code>    |
| §7       | <code>Params.card_openings_le</code>                           | <code>ZkPoP/Privacy.lean</code>    |
| §8       | <code>selfBootTrace_validB</code>                              | <code>ZkPoP/Bundle.lean</code>     |
| §8       | <code>kernelBootstrap_derivable</code>                         | <code>ZkPoP/Bundle.lean</code>     |
| §8       | <code>octra_honest_prover_accepted</code>                      | <code>ZkPoP/Bundle.lean</code>     |
| §8       | <code>octra_bad_index_unqueried</code>                         | <code>ZkPoP/Bundle.lean</code>     |
| Thm. 8.1 | <code>chainDigest_injective</code>                             | <code>ZkPoP/Bundle.lean</code>     |